Week 7: Fast Computation: Prices, Greeks, Implied Vol, Surfaces

The latency budget, in C++ via pybind11 – Milestone 2

North Carolina State University | Summer 2026

NC STATE UNIVERSITY

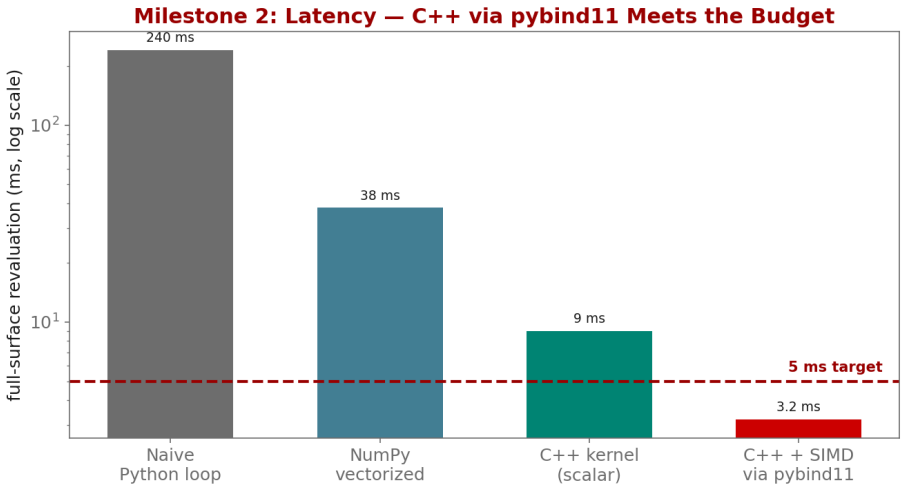
Industry Mentor: Dendi Suhubdy (Bitwyre) | Guest: Fenni Kang (Coincall)

Learning Objectives

- Implement COS / Carr-Madan pricers and robust implied-vol in C++
- Expose the C++ kernels to Python with pybind11
- Validate Greeks against PyTorch autograd; hit the 5 ms budget

- Characteristic functions: Black-Scholes and Heston
- Fang-Oosterlee COS method (the workhorse)
- Robust implied-vol inversion (good guess + safeguarded Newton)

- Structure a pybind11 module; build with CMake
- Pass NumPy/PyTorch tensors with no copy (buffer protocol)
- Release the GIL around the C++ hot loop
- PyTorch autograd is the REFERENCE for Greeks; C++ is the speed


Milestone 2: Latency — C++ via pybind11 Meets the Budget
full-surface revaluation (ms, log scale)
10^2
10^1
240 ms
38 ms
9 ms
5 ms target
3.2 ms
Naive
Python loop
NumPy
vectorized
C++ kernel
(scalar)
C++ + SIMD
via pybind11

Derivation & Rationale

$$C(K) = e^{-rT}\,\frac{1}{\pi}\int_0^{\infty} \mathrm{Re}\big[e^{-iu\ln K}\,\hat{\phi}(u)\big]\,du$$

Why: Once we leave Black-Scholes (e.g. Heston), there is no closed-form price, but the characteristic function ϕ IS known. Pricing becomes a single Fourier integral – this is why CF methods exist and why they are fast.

$$C \approx \sum_{n=0}^{N-1}{}' \mathrm{Re}\Big[\phi\big(\tfrac{n\pi}{b-a}\big)\, e^{-in\pi\frac{a}{b-a}}\Big] V_n$$

Why: The Fang-Oosterlee COS method replaces the integral with a cosine series that converges geometrically for smooth densities. We truncate $[a, b]$ from the cumulants, not by guessing – that is what makes it reach 10^{-4} accuracy in microseconds.

Getting the Data from Coincall

- Inputs are the calibrated surface from Week 6 plus reference prices:
  - `coincall_btc_options_chain_snapshots_2025Q4.parquet`
  - `coincall_calibrated_svi_surfaces_2025Q4.parquet (baseline)`
  - `coincall_reference_prices_2025Q4.parquet (accuracy target, 1e-4)`
- Live mark/IV/Greeks for spot checks: GET /open/option/detail/v1/{symbol}

Cross-check exact paths at https://docs.coincall.com/

Code — fastmm pybind11 module

```
// fastmm.cpp (build with CMake + pybind11; import as `fastmm`)
#include <pybind11/pybind11.h>
#include <pybind11/numpy.h>
namespace py = pybind11;

double cos_price(double S, double K, double T, double r, double sigma); // COS pricer

py::array_t<double> price_surface(py::array_t<double> K, double S,
                                  double T, double r, double sigma) {
    auto k = K.unchecked<1>(); auto out = py::array_t<double>(k.shape(0));
    auto o = out.mutable_unchecked<1>();
    { py::gil_scoped_release nogil; // release GIL for the hot loop
      for (py::ssize_t i = 0; i < k.shape(0); ++i)
          o(i) = cos_price(S, k(i), T, r, sigma); }
    return out; // zero-copy back to NumPy
}
PYBIND11_MODULE(fastmm, m) { m.def("price_surface", &price_surface); }
```

Glossary — Key Terms

Characteristic function the Fourier transform of the log-return density

COS method Fourier-cosine series option pricing (Fang-Oosterlee)

Carr-Madan FFT-based option pricing via a damped payoff

Implied-vol inversion solving for the IV that reproduces a price (safeguarded Newton)

pybind11 header-only C++ <-> Python binding

GIL Python's Global Interpreter Lock; released around the C++ hot loop

Latency budget the maximum allowed time (sub-5 ms full-surface revaluation)

Reading

- Carr-Madan (1999); Fang-Oosterlee (2008); Jaeckel (2015); pybind11 docs

References

- Carr, P., & Madan, D. (1999). Option valuation using the fast Fourier transform. J. Computational Finance, 2(4), 61-73.
- Fang, F., & Oosterlee, C. W. (2008). A novel pricing method for European options based on Fourier-cosine series expansions. SIAM J. Sci. Comput., 31(2), 826-848.
- Jackel, P. (2015). Let's be rational. Wilmott Magazine.
- Jakob, W., Rhinelander, J., & Moldovan, D. (2017). pybind11. https://github.com/pybind/pybind11
- Paszke, A., et al. (2019). PyTorch: an imperative style, high-performance deep learning library. NeurIPS.